

Column Transformer

Module 28 Study Notes • Feature Engineering Pipelines

1. The Problem with Manual Preprocessing

In the previous modules, we learned various feature engineering techniques: Imputing missing values, Ordinal Encoding, One-Hot Encoding, and Feature Scaling. In a real-world dataset, you rarely apply just *one* of these techniques. You typically have to apply **different transformations to different columns simultaneously**.

The "Old Way" (The Headache):

1. Extract column A and apply `SimpleImputer`.
2. Extract columns B and C and apply `OrdinalEncoder`.
3. Extract columns D and E and apply `OneHotEncoder`.
4. Keep column F as it is (no transformation).
5. Use `numpy.concatenate` or `numpy.hstack` to painstakingly stitch all these separate arrays back together into a single master feature matrix (X).

This manual approach is not only tedious and messy, but it is also highly prone to errors, especially when trying to perfectly replicate this exact same process on your Test Data or new production data.

2. The Solution: Scikit-Learn's ColumnTransformer

`ColumnTransformer` is a powerful class from Scikit-Learn's `compose` module. It acts as a **router**. You pass it your entire dataset, and you tell it exactly which columns should go through which transformers. It handles all the isolation, transformation, and final concatenation automatically in the background.

3. Building a ColumnTransformer

To initialize a `ColumnTransformer`, you must pass a list of tuples to the `transformers` parameter. Each tuple represents a specific transformation step and must contain exactly three elements:

The Transformer Tuple: ('name', transformer_object, ['columns'])

- **Name:** A unique string identifier you choose (e.g., 'impute_age', 'ohe_city').
- **Transformer Object:** The actual initialized Scikit-Learn class (e.g., `SimpleImputer()`).
- **Columns:** A list of the column names (strings) or column indices (integers) to apply the transformer to.

Handling the Leftovers: The `remainder` Parameter

When you define your transformers, you usually don't specify every single column in your dataset. What happens to the columns you didn't mention?

- `remainder='drop'` (Default): Any column not explicitly mentioned in the transformer list is deleted from the final output array.
- `remainder='passthrough'` : Any column not mentioned is left entirely untouched and simply appended as-is to the final output array. (This is almost always what you want to use).

4. Implementation Workflow

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder

# 1. Train-Test Split (Always step 1!)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. Define the ColumnTransformer
transformer = ColumnTransformer(
    transformers=[
        # Tuple: (name, object, columns)
        ('impute_fever', SimpleImputer(), ['fever']),
        ('ordinal_cough', OrdinalEncoder(categories=[['Mild', 'Strong']])),
        ['cough']),
        ('ohe_city', OneHotEncoder(drop='first', sparse=False), ['gender',
        'city'])
    ],
    remainder='passthrough' # Keep 'Age' column untouched
)
```

```
# 3. Fit & Transform in one clean step!
X_train_transformed = transformer.fit_transform(X_train)

# 4. Replicate on test data effortlessly
X_test_transformed = transformer.transform(X_test)
```

5. Key Takeaways

- **Clean Code:** What took 20+ lines of messy array slicing and concatenation is reduced to a single elegant block.
- **Production Ready:** The `transformer` object "remembers" everything (e.g., the imputed mean, the OHE categories). When you deploy your model, you just save this single object and call `.transform()` on new incoming user data.